COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE 010 - Data Structure & Algorithms | **SECTION:** | CPE21S4 |
| **DATE PERFORMED:** | Oct 2, 2026 | **DATE SUBMITTED:** | Oct 6, 2026 |
| **NAME:** | Bal, Kevin Ranz J. | **INSTRUCTOR:** | Engr. Jimlord M. Quejado |

## Hands-on Activity 10.1 Graphs

### 1. PROCEDURE OUTPUT

What procedures did you perform?

| **PROCEDURE:** |
|---|
| **Procedure that performed:** |
| |
| **ILO A.3** |
| **Task Title & Task Objective** |
| |

**Screenshot of the code**

CPE21S4_BAL > Finals > Hands-on Activities > HoA 10.1 > ILO A > main.cpp > ...

```cpp
#include <iostream>

// stores adjacency list items
struct adjNode
{
    int val, cost;
    adjNode *next;
};

// structure to store edges
struct graphEdge
{
    int start_ver, end_ver, weight;
};

class DiaGraph
{
    // insert new nodes into adjacency list from given graph
    adjNode *getAdjListNode(int value, int weight, adjNode *head)
    {
        adjNode *newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;
        newNode->next = head; // point new node to current head
        return newNode;
    }

    int N; // number of nodes in the graph
public:
    adjNode **head; // adjacency list as array of pointers
    // Constructor
    DiaGraph(graphEdge edges[], int n, int N)
    {

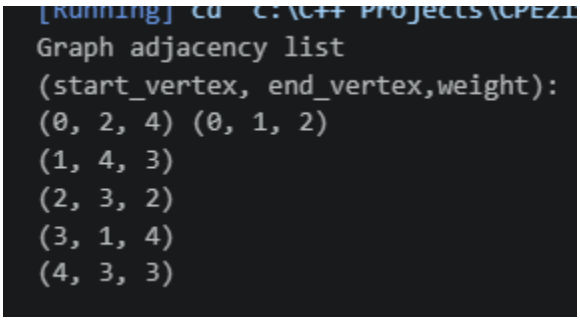
        // allocate new node
        head = new adjNode *[N]();
        this->N = N;
        // initialize head pointer for all vertices
        for (int i = 0; i < N; ++i)
            head[i] = nullptr;
        // construct directed graph by adding edges to it
        for (unsigned i = 0; i < n; i++)
        {
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            int weight = edges[i].weight;
            // insert in the beginning
            adjNode *newNode = getAdjListNode(end_ver, weight, head[start_ver]);
            // point head pointer to new node
            head[start_ver] = newNode;
        }
    }

```

```
    // Destructor
    ~DiaGraph()
    {
        for (int i = 0; i < N; i++)
        {
            delete[] head[i];
        }
        delete[] head;
    }
};

// print all adjacent vertices of given vertex
void display_AdjList(adjNode *ptr, int i)
{
    while (ptr != nullptr)
    {
        std::cout << "(" << i << ", " << ptr->val << ", " << ptr->cost << ") ";
        ptr = ptr->next;
    }
    std::cout << std::endl;
}

// graph implementation
int main()
{
    // graph edges array.
    graphEdge edges[] = {
        // (x, y, w) -> edge from x to y with weight w
        {0, 1, 2},
        {0, 2, 4},
        {1, 4, 3},
        {2, 3, 2},
        {3, 1, 4},
        {4, 3, 3}};
    int N = 6; // Number of vertices in the graph
    // calculate number of edges
    int n = sizeof(edges) / sizeof(edges[0]);
    // construct graph
    DiaGraph diagraph(edges, n, N);
    // print adjacency list representation of graph
    std::cout << "Graph adjacency list " << std::endl
              << "(start_vertex, end_vertex,weight):" << std::endl;
    for (int i = 0; i < N; i++)
    {
        // display adjacent vertices of vertex i
        display_AdjList(diagraph.head[i], i);
    }
    return 0;
}
```

| | |
|---|---|
| **Output of the code** |  |
| **Analysis** | **This file uses pointers and custom node structures to build a directed graph configuration. It connects nodes dynamically by changing pointer targets, which uses memory efficiently because it only allocates space. The print function loops through these pointers to display clear edge details and connection on the screen.** |

| |
|---|
| **ILO B.1** |

| **Task Title & Task Objective** |
|---|
| **Depth-First Search** |

**Header DFS**

CPE21S4_BAL > Finals > Hands-on Activities > HoA 10.1 > ILO B.1 > C DFS.h > ...

```
// step 1
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

// step 2
template <typename T>
class Graph_DFS;

// step 2 (part 2)
template <typename T>
struct Edge_DFS
{
    size_t src;
    size_t dest;
    T weight;

    // To compare edges, only compare their weights,
    // and not the source/destination vertices
    inline bool operator<(const Edge_DFS<T> &e) const
    {
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge_DFS<T> &e) const
    {
        return this->weight > e.weight;
    }
};

// step 3
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph_DFS<T> &G);

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph_DFS<T> &G)
{

    for (auto i = 1; i <= G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
        {
            os << "{" << e.dest << ": " << e.weight << "}, ";
        }
        os << std::endl;
    }
    return os;
```

```
    return os;
}

// step 4
template <typename T>
class Graph_DFS
{

private:
    size_t V; // Stores number of vertices in graph
    std::vector<Edge_DFS<T>> edge_list;

public:
    // Initialize the graph with N vertices
    Graph_DFS(size_t N) : V(N) {}
    void add_edge(Edge_DFS<T> &&e);
    auto outgoing_edges(size_t v) const;

    // Return number of vertices in the graph
    auto vertices() const
    {
        return V;
    }

    // Return all edges in the graph
    auto &edges() const
    {
        return edge_list;
    }

    // step 4 (part 2)
    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph_DFS<U> &G);
};

// step 4 (part 2)
template <typename T>
void Graph_DFS<T>::add_edge(Edge_DFS<T> &&e)
{
    // Check if the source and destination vertices are within range
    if (e.src >= 1 && e.src <= V && e.dest >= 1 && e.dest <= V)
    {
        edge_list.emplace_back(e);
    }
    else
    {
        std::cerr << "Vertex out of bounds" << std::endl;
    }
}
//  Returns all outgoing edges from vertex v

// step 4 (part 3)
```

```cpp
// step 4 (part 3)
template <typename T>
auto Graph_DFS<T>::outgoing_edges(size_t v) const
{
    std::vector<Edge_DFS<T>> edges_from_v;
    for (auto &e : edge_list)
    {
        if (e.src == v)
        {
            edges_from_v.emplace_back(e);
        }
    }
    return edges_from_v;
}
// Overloads the << operator so a graph be written directly to a stream
// Can be used as std::cout << obj << std::endl;

/* removed cause of duplicate definition of Graph class
// step 4 (part 4)
template <typename T>
class Graph{
    private:
        size_t V; // Stores number of vertices in graph
        std::vector<Edge<T>> edge_list;

    public:
        friend std::ostream &operator<<(std::ostream &os, const Graph<T> &G);
    };
*/
```

```cpp
// step 4 (part 4)
template <typename T>
auto depth_first_search_DFS(const Graph_DFS<T> &G, size_t dest)
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    stack.push(1); // Assume that DFS always starts from vertex ID 1
    while (!stack.empty())
    {
        auto current_vertex = stack.top();
        stack.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
            {
                // If the vertex hasn't been visited, insert it in the stack.
                if (visited.find(e.dest) == visited.end())
                {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}

// step 5
// implementation of DFS
// debug
template <typename T>
auto create_reference_graph_DFS()
{

    Graph_DFS<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};
```

```
    for (auto &i : edges)
    {
        for (auto &j : i.second)
        {
            G.add_edge(Edge_DFS<T>{i.first, j.first, j.second});
        }
    }
    return G;
}

// step 6
// run DFS
template <typename T>
void test_DFS()
{
    // Create an instance of and print the graph

    auto G = create_reference_graph_DFS<unsigned>();
    std::cout << G << std::endl;

    // Run DFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search_DFS(G, 1);

    for (auto v : dfs_visit_order)
    {
        std::cout << v << std::endl;
    }
}
```

| | |
|---|---|
| **Main Driver DFS** | CPE21S4_BAL > Finals > Hands-on Activitie<br>1 #include <iostream><br>2 #include <string><br>3 #include <vector><br>4 #include <set><br>5 #include <map><br>6 #include <queue><br>7 #include "DFS.h"<br>8<br>9 int main()<br>10 {<br>11 using T = unsigned;<br>12 test_DFS<T>();<br>13<br>14 return 0;<br>15 } |
| **Output** | 1: {2: 0}, {5: 0},<br>2: {1: 0}, {5: 0}, {4: 0},<br>3: {4: 0}, {7: 0},<br>4: {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},<br>5: {1: 0}, {2: 0}, {4: 0}, {8: 0},<br>6: {4: 0}, {7: 0}, {8: 0},<br>7: {3: 0}, {6: 0},<br>8: {4: 0}, {5: 0}, {6: 0},<br>9:<br><br>DFS Order of vertices:<br>1<br>5<br>8<br>6<br>7<br>3<br>4<br>2 |
| **Analysis** | **This DFS code uses a stack to explore a path as deep before backtracking to check other branches. The logic separates the edge struct and overloads the output operator to print node structures neatly. A structural problem** |

| | |
|---|---|
| | remains because storing edges in a single flat list forces the program to read through unrelated connections during neighbor lookups. Furthermore, because a stack uses a last-in, first-out method (LIFO), the revealed traversal path depends on the order edges are added to the graph. Just like the BFS header, this section uses a friend template header (template <typename U>) to grant the printing function global access to private data bounds without creating type compilation locks. |

| ILO B.2 |
|---|

| Task Title & Task Objective |
|---|
| |

**Header BFS**

CPE21S4_BAL > Finals > Hands-on Activities > HoA 10.1 > ILO B.2 > C BFS.h > Edge_BFS<T>

```
// step 1
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

// step 2
template <typename T>
class Graph_BFS;

// step 2 (part 2)
template <typename T>
struct Edge_BFS
{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge_BFS<T> &e) const
    {
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge_BFS<T> &e) const
    {
        return this->weight > e.weight;
    }
};

// step 3
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph_BFS<T> &G)
{
    for (auto i = 1; i <= G.vertices(); i++)
    {
        os << i << ":\t";

        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
        {
            os << "{" << e.dest << ": " << e.weight << "}, ";
        }
        os << std::endl;
    }
    return os;
}

// step 4
template <typename T>
class Graph_BFS
```

```cpp
// step 4
template <typename T>
class Graph_BFS
{

private:
    size_t V;
    std::vector<Edge_BFS<T>> edge_list;

public:
    Graph_BFS(size_t N) : V(N) {}
    auto outgoing_edges(size_t v) const;
    void add_edge(Edge_BFS<T> &&e);

    auto vertices() const
    {
        return V;
    }

    auto &edges() const
    {
        return edge_list;
    }

    // step 4 (part 2)
    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph_BFS<U> &G);
};

// step 4 (part 3)
template <typename T>
void Graph_BFS<T>::add_edge(Edge_BFS<T> &&e)
{
    if (e.src >= 1 && e.src <= V && e.dest >= 1 && e.dest <= V)
    {
        edge_list.emplace_back(e);
    }
    else
    {
        std::cerr << "Vertex out of bounds" << std::endl;
    }
}

// step 4 (part 4)
template <typename T>
auto Graph_BFS<T>::outgoing_edges(size_t v) const
{
```

```
void Graph_BFS<T>::add_edge(Edge_BFS<T> &&e)

// step 4 (part 4)
template <typename T>
auto Graph_BFS<T>::outgoing_edges(size_t v) const
{
    std::vector<Edge_BFS<T>> edges_from_v;
    for (auto &e : edge_list)
    {
        if (e.src == v)
        {
            edges_from_v.emplace_back(e);
        }
    }
    return edges_from_v;
}

/* removed cause of duplicate definition of Graph class
// step 4 (part 2)
template <typename T>
class Graph{
friend std::ostream &operator<<(std::ostream &os, const Graph<T> &G);

    private:
    size_t V;
    std::vector<Edge<T>> edge_list;
}
*/

// step 5
// BSF implementation
template <typename T>
auto create_reference_graph_BFS()
{
    Graph_BFS<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};
    for (auto &i : edges)
    {
        for (auto &j : i.second)
        {
            G.add_edge(Edge_BFS<T>{i.first, j.first, j.second});
        }
    }
    return G;
```

```
    return G;
}

// step 6
template <typename T>
auto breadth_first_search_BFS(const Graph_BFS<T> &G, size_t dest)
{

    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    queue.push(1); // Assume that BFS always starts from vertex ID 1
    while (!queue.empty())
    {
        auto current_vertex = queue.front();
        queue.pop();

        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex))
            {
                queue.push(e.dest);
            }
        }
    }
    return visit_order;
}

// step 7
template <typename T>
void test_BFS()
{
    // Create an instance of and print the graph
    auto G = create_reference_graph_BFS<unsigned>();
    std::cout << G << std::endl;

    // Run BFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search_BFS(G, 1);
    for (auto v : bfs_visit_order)
    {
        std::cout << v << std::endl;
    }
}
```

| | |
|---|---|
| **Main Driver BFS** | CPE21S4_BAL > Finals > Hands-on Ac<br>1 #include <iostream><br>2 #include <string><br>3 #include <vector><br>4 #include <set><br>5 #include <map><br>6 #include <queue><br>7 #include "BFS.h"<br>8<br>9 int main()<br>10 {<br>11 using T = unsigned;<br>12 test_BFS<T>();<br>13<br>14 return 0;<br>15 } |
| **Output of the code** | 1: {2: 2}, {5: 3},<br>2: {1: 2}, {5: 5}, {4: 1},<br>3: {4: 2}, {7: 3},<br>4: {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},<br>5: {1: 3}, {2: 5}, {4: 2}, {8: 3},<br>6: {4: 4}, {7: 4}, {8: 1},<br>7: {3: 3}, {6: 4},<br>8: {4: 5}, {5: 3}, {6: 1},<br>9:<br><br>BFS Order of vertices:<br>1<br>2<br>5<br>4<br>8<br>3<br>6<br>7 |
| **Analysis** | **This BFS file creates a graph structure and uses a queue to visit vertices layer by layer. The search correctly prevents endless loops by tracking already visited numbers in a set data container. However, using basic flat vectors means the search must scan the entire edge list to find** |

| | |
|---|---|
| | **neighbors, which can slow down operations on large networks. Because a queue processes nodes in a first-in, first-out order, this setup guarantees finding the shortest path in terms of total edges for unweighted layouts. Additionally, the class structure includes an independent friend template identifier (template <typename U>).** |

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

**SUPPLEMENTARY:**

**ILO C**

**Task Title & Objective:**

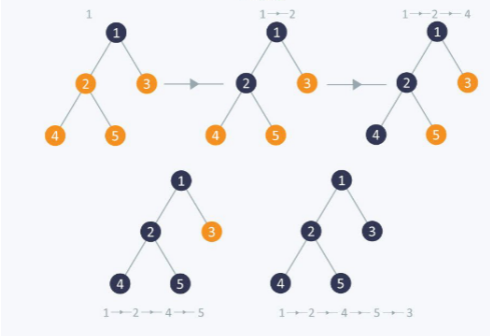
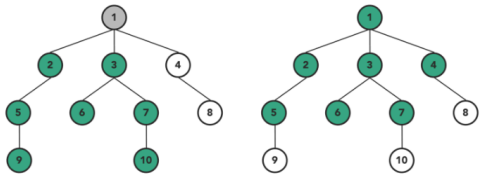
1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from the same vertex. Discuss which algorithm would be most helpful to accomplish this task.

- **The Depth First Search (DFS) algorithm is the most helpful for this task. DFS works by choosing a path and traveling as far down that line as possible until it cannot go any further. Once it searches and travels the deepest and hits the last node, it automatically backtracks to the previous intersection and starts exploring the next available path from that same spot. This matches the exact map-visiting behavior you described because it prioritizes depth over width before heading back.**

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

- **In trees, the equivalent of DFS is found in the preorder, inorder, and postorder traversal strategies. Just like DFS, these tree strategies go down a single path until all the lower leaf nodes are traversed, and then they backtrack to visit the remaining unvisited branches from a higher level. Both concepts use a Stack data structure that follows the Last In, First Out rule to handle this deep movement. The main difference is that tree strategies aim to visit every single node in a strictly orderly manner, while a general graph DFS simply focuses on exploring as deep as possible down a path until it can no longer move forward.**

# Comparison of the Depth-First Search (DFS) and Traversal Strategies

| Depth-First Search (DFS) | Traversal Strategies |
| --- | --- |
|  |  |

**Pseudocode**

```
START

FUNCTION preOrder

INPUT leftChildNode
INPUT rightChildNode
INPUT key

DISPLAY key
IF leftChildNode is not equal to nullptr
 leftChildNode points to preOrder
IF rightCHildNode is not equal to nullptr
 rightChildNode points to preOrder

END

START

FUNCTION inOrder

INPUT leftChildNode
INPUT rightChildNode
INPUT key
```

```
IF leftChildNode is not equal to nullptr
 leftChildNode points to inOrder
DISPLAY key
IF rightCHildNode is not equal to nullptr
 rightChildNode points to inOrder

END

START

FUNCTION postOrder

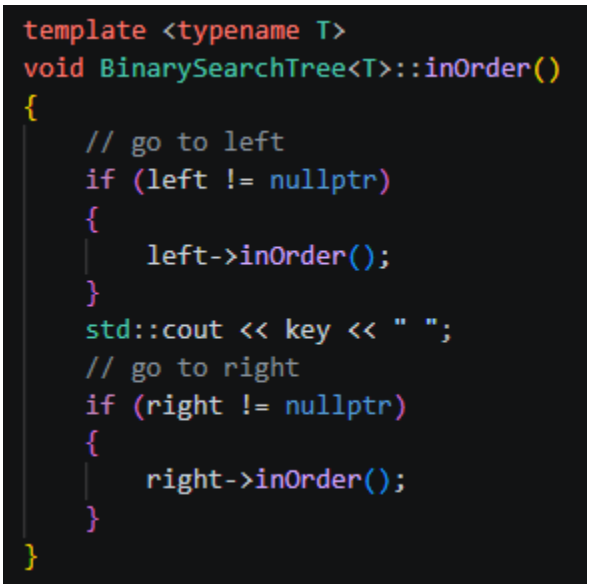
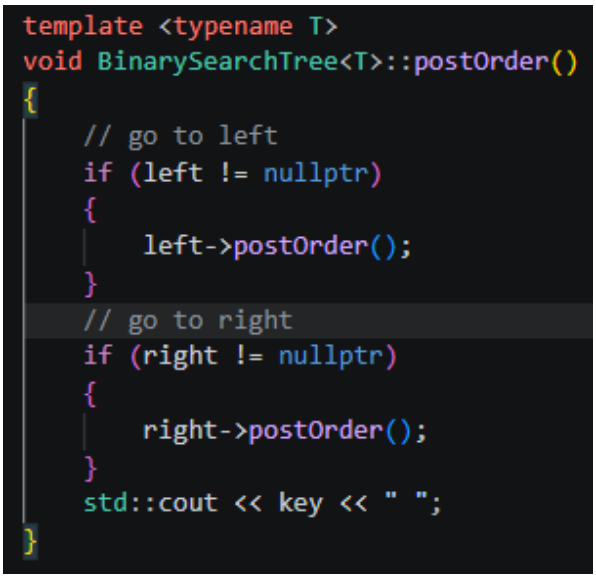
INPUT leftChildNode
INPUT rightChildNode
INPUT key

IF leftChildNode is not equal to nullptr
 leftChildNode points to postOrder
IF rightCHildNode is not equal to nullptr
 rightChildNode points to postOrder
DISPLAY key

END
```

| | |
|---|---|
| preOrder Traversal | |

```cpp
template <typename T>
void BinarySearchTree<T>::preOrder()
{
    std::cout << key << " ";
    if (left != nullptr)
    {
        left->preOrder();
    }
    if (right != nullptr)
    {
        right->preOrder();
    }
}
```

| | |
|---|---|
| inOrder Traversal |  |
| postOrder Traversal |  |

```
template <typename T>
void BinarySearchTree<T>::inOrder()
{
    // go to left
    if (left != nullptr)
    {
        left->inOrder();
    }
    std::cout << key << " ";
    // go to right
    if (right != nullptr)
    {
        right->inOrder();
    }
}
```

```
template <typename T>
void BinarySearchTree<T>::postOrder()
{
    // go to left
    if (left != nullptr)
    {
        left->postOrder();
    }
    // go to right
    if (right != nullptr)
    {
        right->postOrder();
    }
    std::cout << key << " ";
}
```

3. In the performed code, what data structure is used to implement the Breadth First Search?
   - **In the provided code, a standard queue (std::queue) is the primary data structure used to implement the Breadth First Search. A queue works on a first-in, first-out rule, meaning the first location added is the first one processed. This specific method forces the algorithm to explore the graph evenly layer by layer, moving outward to close neighbors before exploring farther distances.**

4. How many times can a node be visited in the BFS?

- **In a Breadth-First Search algorithm, nodes can only be visited once. Because BFS is a traversing algorithm that moves layerwise, you must fully explore all the neighboring nodes on your current level before you can proceed to any next-level neighbor nodes. This layer-by-layer progression means the algorithm only moves forward and completely lacks the ability to backtrack to previous nodes like a Depth-First Search does.**

**Conclusion**

**This activity provided me with great practice, analysis and demonstration in identifying and fixing structural template errors that were given to us and including demonstrating visually of how DFS, BFS, Adjacency List and Matrix Representation, fixing the multiple lines of error codes in both header on purpose, and broken loop nesting in Hands-on Activity 10.1 Graph. Working through the files showed how a queue drives the wide, layer-by-layer discovery of Breadth First Search, while a stack manages the deep path-tracking and backtracking behavior of Depth First Search. Comparing these graph algorithms to tree traversals made it clear that while tree strategies focus on a strict, orderly lookup, graph searches are designed to navigate complex networks safely using tracking sets to avoid infinite loops.**

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **Google Search**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **Explaining the concepts, and code assistance**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **Used to explain how does the each unfamiliar functions work and behave on both header and what does it do and telling that explain and not to fix it, including the assistance of using the template<typename U> and how does it work and when to use it**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.




I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.